학생용 · 취업 준비 자료

COMET FOUNDRY

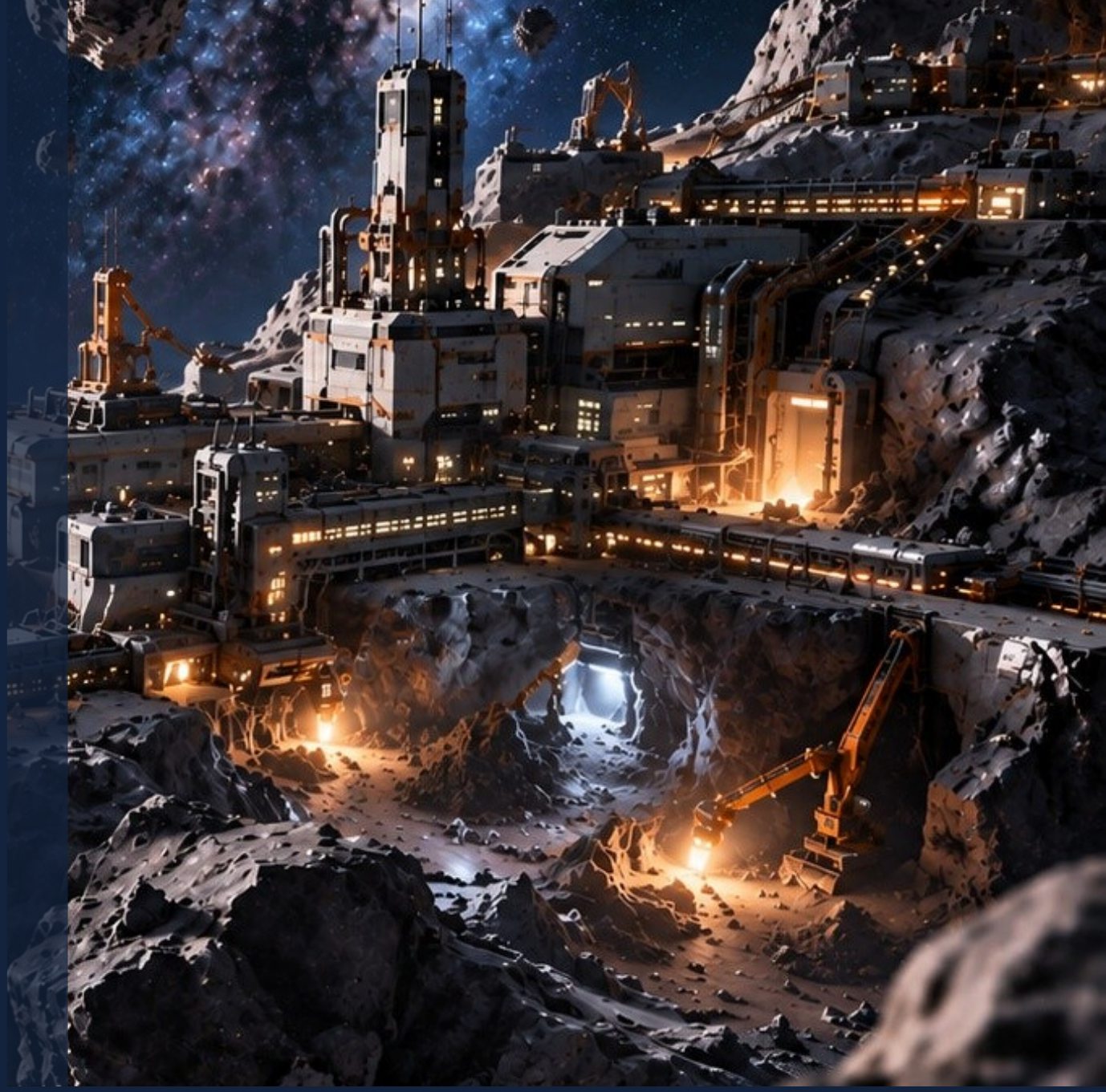
게임 프로그래머 취업 포트폴리오 준비 가이드

게임 기능을 소개하는 데서 끝내지 말고 ,
내가 작성한 코드와 검증 과정까지 보여주세요 .

핵심 메시지

“무엇을 만들었는가” + “ 어떻게 만들었는가” + “ 정말 잘 작동하는가”

기준 프로젝트 : 음승재 · Comet Foundry



포트폴리오의 핵심은 딱 3 가지입니다

처음 보는 개발자도 “무엇을 직접 했고, 어떤 기술이 있으며, 검증했는지” 빠르게 파악하게 만드세요.

01

내가 직접 만든 것

어떤 기능을 맡았는지
직접 기여한 범위를 구분

기여 범위

02

코드가 해결하는 문제

어떤 규칙과 예외를
어떤 구조로 처리했는지 설명

설계 · 코드

03

검증한 근거

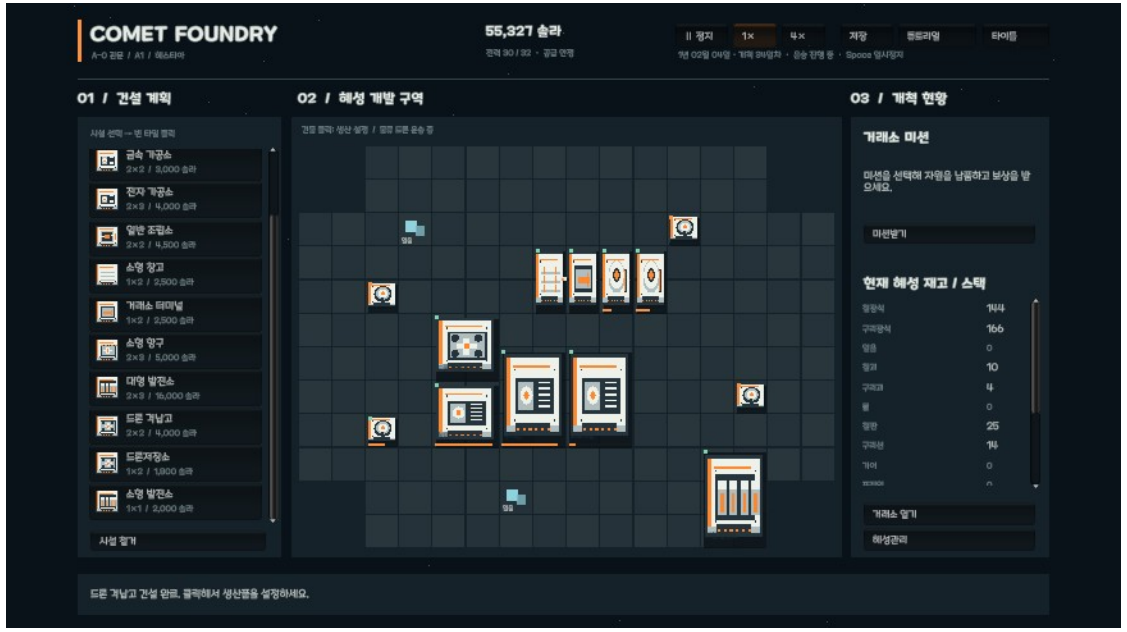
테스트 · 성능 측정으로
결과를 확인

테스트 결과

좋은 포트폴리오 = 실제 구현 + 이해하기 쉬운 설명 + 확인 가능한 증거

Comet Foundry 는 무엇을 보여줄 수 있나요 ?

생산 · 물류 · 거래처럼 서로 영향을 주는 시스템을 연결한 2D 산업 경영 시뮬레이션입니다 .



실제 프로젝트 화면

게임 안에서 자원을 모으고 , 가공하고 , 옮기고 , 거래합니다 .

※ 플레이 화면은 업로드된 프로젝트 자료에서 가져왔습니다 .

생산 레시피 · 원재료 · 전력 · 생산 시간

물류 자동 드론 · 수송선 · 도착 공간

경영 구매 · 판매 · 자금 · 혜성 확장

안정성 저장 · 복원 · 철거 · 미션

추천 핵심 사례 : 생산 · 물류 / 수송 · 저장 / 복원

좋은 점은 살리고 , 부족한 증거를 채우세요

기능을 더 추가하는 것보다 기존 기능을 코드와 테스트로 설명하는 일이 우선입니다 .

이미 갖춘 장점

- ✓ 생산 · 물류 · 거래 등 여러 시스템이 연결되어 있다
- ✓ 실제 인게임 화면과 기능 설명이 준비되어 있다
- ✓ 저장 · 복원과 예외 처리처럼 기술 사례가 다양하다
- ✓ 테스트 코드 및 Windows 빌드 기록이 자료에 언급되어 있다

반드시 보완할 부분

- ! 직접 작성한 C# 핵심 코드가 보이지 않는다
- ! 본인이 구현한 범위와 외부 도움 범위를 구분해야 한다
- ! 실제 실행한 테스트 결과가 필요하다
- ! 성능 측정 수치와 개선 전 · 후 근거가 없다
- ! GitHub · README · 실행 방법이 한곳에 정리되어야 한다

중요 : 코드와 테스트는 실제 프로젝트에서 확인한 내용만 기재하세요 .

포트폴리오는 이 순서로 구성하세요

면접관이 게임을 이해한 다음 , 기술적 깊이와 검증 수준을 확인하게 만드는 흐름입니다 .

1

프로젝트 한눈에 보기

장르 · 기술 스택 · 실행 링크

2

시스템 구조

주요 클래스와 데이터 흐름

3

핵심 기술 사례 3 개

문제 → 설계 → 코드 → 테스트

4

테스트 · 성능 검증

실제 결과와 개선 근거

5

제출 자료 · 회고

GitHub · README · 한계와 계획

기존 23 페이지 자료를 전부 버릴 필요는 없습니다 .

버튼별 기능 소개는 압축하고 , 코드 · 테스트 · 개선 사례에 공간을 더 주세요 .

기술서는 “코드 목록”이 아니라 “문제 해결 기록”입니다

한 시스템을 선택하고 아래 6 단계로 설명하면 됩니다 .

1 문제 정의

어떤 문제를 해결해야 했나 ?

2 규칙 · 조건

반드시 지켜야 할 규칙은 ?

3 구조 설계

어떤 클래스가 역할을 맡나 ?

4 핵심 코드

중요한 메서드는 어떻게 동작하나 ?

5 테스트

정상 · 실패 · 경계 조건을 확인했나 ?

6 결과 · 개선

무엇이 해결됐고 다음은 무엇인가 ?

각 단계에 실제 코드 , 그림 , 테스트 결과를 붙여서 설명하세요 .

생산 기능은 “재료를 소비하고 결과를 만드는 규칙”을 설명하세요

프로젝트 자료에 있는 레시피 · 자원 · 전력 · 생산 진행도 처리를 기술 사례로 선택할 수 있습니다 .



실제 플레이 화면

기술서에서 확인할 질문

- 재료 · 전력 · 저장 공간은 언제 검사하는가 ?
- 생산 도중 레시피가 바뀌면 진행도는 어떻게 되는가 ?
- 완료 전후 재고가 정확히 바뀌는가 ?

생산 처리 흐름 예시

생산 요청

↓

조건 검사

↓

진행도 갱신

↓

완성품 반영

테스트 예 : 재료 부족 / 전력 부족 / 레시피 변경

※ 흐름도는 설명용입니다 . 실제 코드 처리 순서를 확인해 수정하세요 .

물류 시스템은 “이동하는 동안 데이터가 어떻게 유지되는가”를 보여주세요

드론 운반과 혜성 간 수송을 구분하고, 화물이 손실되거나 중복 처리되지 않는지 설명합니다.

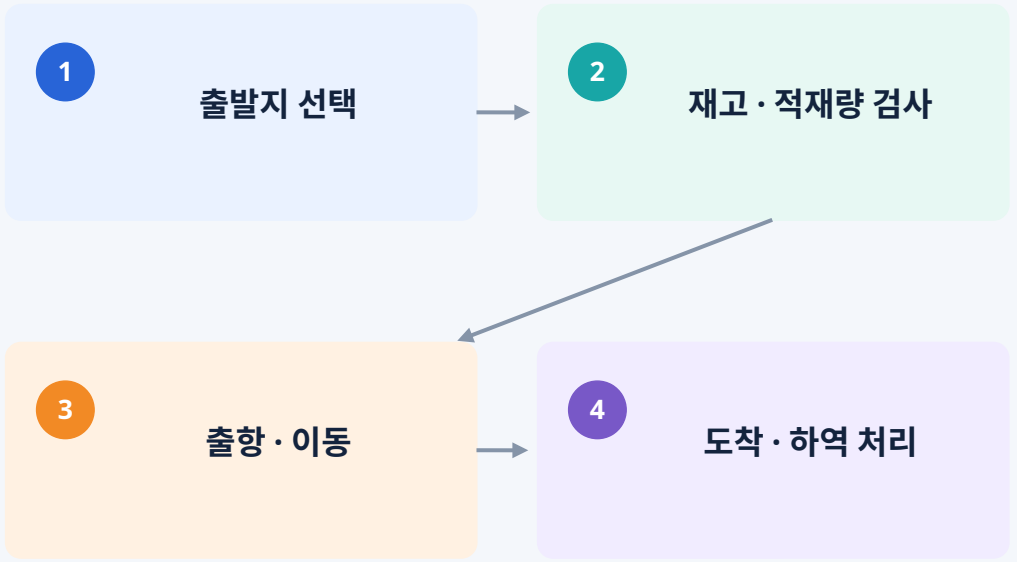


화면은 예시 자료입니다. 물류 UI에 맞는 실제 화면을 골라 넣으세요.

기술서에 답할 질문

- 출발할 때 화물 재고는 어떻게 처리되는가?
- 도착 항구가 가득 차면 화물은 어디에 남는가?
- 일시정지 · 저장 · 복원에도 상태가 유지되는가?

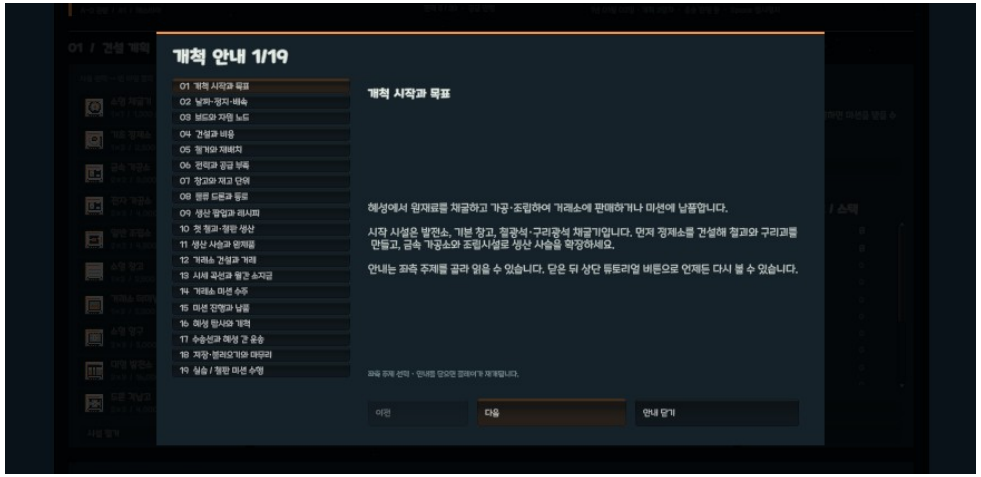
수송 흐름



핵심 검증: 화물 유실 없음 · 중복 지급 없음

저장 시스템은 “다시 불러와도 상태가 같은가”가 핵심입니다

현재 프로젝트 자료에 소개된 저장 데이터와 오류 처리 규칙을 테스트로 증명하세요 .



관련 화면 또는 실제 저장 · 복원 테스트 화면으로 교체

저장 전후 비교할 데이터

- 보유 혜성 및 자원 수량
- 시설의 생산품과 진행도
- 수송 중 선박과 화물
- 현재 미션 · 진행 상태

권장 테스트 흐름

1. **게임 상태 만들기**
   생산 중 / 수송 중
2. **저장 실행**
   저장 데이터 확인
3. **게임 재실행 후 복원**
   같은 상태인지 비교
4. **손상 데이터도 확인**
   안전하게 거부하는지

좋은 결과 기록 : 저장 전 · 후 값과 테스트 결과

코드는 짧게 보여주고 , 중요한 줄을 설명하세요

아래 코드는 설명 방법을 보여주는 예시입니다 . 기존 프로젝트에 실제로 존재하는 코드라는 뜻은 아닙니다 .

설명용 의사코드 · 실제 코드로 교체

```
bool CanStartProduction(Recipe recipe)
{
    if (recipe == null) return false;

    // 생산 전에 필요한 조건을 먼저 확인
    if (!HasRequiredPower(recipe)) return false;
    if (!HasRequiredMaterials(recipe)) return false;
    if (!HasStorageSpace(recipe)) return false;

    return true;
}
```

※ 실제 메서드명 · 검사 순서는 원본 코드에 맞게 바꾸세요 .

코드 옆에는 이런 설명을 붙이세요

무엇을 검사하나 ?	레시피 , 전력 , 재료 , 저장 공간
왜 먼저 검사하나 ?	불가능한 생산이 시작되지 않게 하기 위해
실패하면 어떻게 되나 ?	생산 상태나 재고가 잘못 바뀌지 않아야 함
어떻게 확인하나 ?	재료 부족 · 전력 부족 테스트 실행

테스트는 “실제로 실행한 결과”를 남겨야 합니다

테스트 코드를 작성한 것과 테스트가 통과한 것은 다릅니다. 실행 날짜와 결과 화면을 함께 기록하세요.

시스템	테스트 조건	기대 결과 (실제 규칙과 맞춰 수정)
생산	재료가 부족하다	생산이 시작되지 않음 / 자원 값 유지
전력	전력이 부족하다	규칙에 따라 생산 중단
거래	구매 자금이 부족하다	거래가 거부되고 자금 · 재고 유지
물류	도착 항구 공간이 부족하다	화물이 유실되지 않고 대기
저장	수송 중 저장 후 복원	선박 · 화물 · 목적지가 올바르게 복원
철거	창고 공간이 부족한 시설 철거	철거가 거부되고 재고 보존

증빙으로 남길 것 : 테스트 코드 + 실행 결과 캡처 + 실패 원인 / 수정 내역

최적화했다고 말하려면 , 측정값이 있어야 합니다

시설과 혜성이 늘어날 때 병목이 생기는지 확인하고 , 개선 전후를 같은 환경에서 비교하세요 .

측정 절차

1. 같은 PC 와 같은 빌드로 시작한다
2. 시설 · 혜성 수를 단계별로 늘린다
3. Profiler 에서 병목 구간을 찾는다
4. 코드를 수정하고 같은 조건으로 재측정한다

추천 측정 항목

FPS / 프레임 시간	화면이 얼마나 부드러운가
CPU 사용 시간	게임 로직 중 어디가 느린가
GC Alloc	프레임마다 메모리를 얼마나 할당하는가
메모리 사용량	오브젝트 · 데이터 증가에 따라 얼마나 커지는가

수치가 없다면 개선율을 만들어 쓰지 마세요 .

포트폴리오 PDF 만 보내지 말고 , 실행과 코드도 함께 제공하세요

담당자가 바로 열어보고 이해할 수 있도록 링크와 문서를 정리합니다 .

CV 이력서

지원 직무 · 기술 · 프로젝트

PDF 포트폴리오 PDF

문제 · 설계 · 코드 · 검증

▶ 실행 가능한 게임

Unity Play 링크 또는 빌드

REC 플레이 영상

1~3 분 내외 핵심 플레이

<> GitHub 저장소

직접 작성한 코드와 구조

MD README

실행 · 조작 · 버전 · 폴더 구조

C# 스크립트 기술서

핵심 메서드와 처리 흐름

✓ 테스트 · 성능 자료

실행 결과와 측정 수치

README 에 꼭 넣기 Unity 버전 · 실행 방법 · 조작법 · 폴더 구조 · 테스트 방법 · 알려진 문제 · 외부 에셋 출처

프로젝트 전체 공개가 어렵다면 , 공개 가능한 핵심 코드와 별도 설명 문서만 제공해도 됩니다 .

현재 프로젝트는 어떤 직무와 잘 연결될까요 ?

현재 자료에 나타난 시스템 개발 경험을 기준으로 정리했습니다 . 직무별 요구 기술은 회사 공고에 맞춰 추가하세요 .

Unity 클라이언트 프로그래머	게임 상태 · UI 연동 · 입력 처리 · 데이터 관리	추천
게임플레이 · 시스템 프로그래머	생산 규칙 · 자원 흐름 · 거래 · 상태 전이 · 예외 처리	추천
서버 · 네트워크 프로그래머	네트워크 통신 · 서버 권위 구조 · 동시성 · DB 증거 필요	추가 증명
엔진 · 그래픽스 프로그래머	렌더링 · 셰이더 · 엔진 내부 · 성능 최적화 증거 필요	추가 증명

우선 추천 : Unity 클라이언트 / 게임플레이 · 시스템 프로그래머

아래 5 단계만 차례대로 준비하세요

새로운 기능을 무작정 추가하기보다 , 이미 만든 기능을 직접 설명하고 검증하는 데 집중하세요 .

1. **직접 작성한 코드 목록 만들기** — 직접 구현 / 수정 / 외부 코드 · 도움 범위를 구분
2. **기존 테스트 실행하기** — Unity Test Runner 결과와 실패 원인을 기록
3. **핵심 기술서 3 개 작성하기** — 생산 → 물류 · 수송 → 저장 · 복원 순서
4. **실제 성능과 플레이 확인하기** — Profiler 측정 및 다른 사람이 플레이한 피드백
5. **최종 자료 묶기** — PDF + 실행 링크 + GitHub + README + 영상

기억하세요 : 기능의 개수보다 , 문제 해결 과정과 실제 검증이 더 중요합니다 .